

PipeMaster v2 Tutorial

Demographic Inference from RAD-seq and WGS Data

Contents

Contents	2
1. Installation	2
Simulation engines	2
Parameter scaling	3
2. Example Models	3
2.1 Vaquita two-epoch bottleneck	3
2.2 Orangutan isolation with migration	3
2.3 Human Out-of-Africa	4
Loading example data	4
3. Computing Observed Statistics	5
From a VCF file	5
Multi-population example (PonAbe)	5
Using pre-computed WGS observed stats	6
Suggesting prior bounds from observed stats	6
4. Simulating Reference Tables	7
RAD-seq reference table (msABC engine)	7
WGS reference table (scrm engine)	7
Loading a reference table	7
Simulation budget guidelines	8
5. Supervised Machine Learning	8
Step 1: Hyperparameter search with <code>tune.nn()</code>	8
Step 2: Prediction with <code>nn.predict()</code>	8
Step 3: Plot posteriors	9
Loading saved results	9
6. Out-of-Distribution Diagnostics	9
Pre-training check (no NN required)	10
Post-training check (requires trained NN)	10
Drilling into outliers	10
Diagnosing prior bounds	10
Using OOD to filter statistics	11
7. Worked Example: PonAbe (Orangutan) RAD-seq, End to End	11
7.1 Load the model and attach the data	11
7.2 Observed statistics	12
7.3 Column bookkeeping	12
7.4 Reference table	12
7.5 Prior check before you spend compute	13
7.6 Train and predict	13
7.7 Post-training diagnostics	14
8. Model Selection: PonAbe IM vs Strict Isolation	14
8.1 Build the competing model	14
8.2 Simulate the second reference table	14
8.3 Stack into a labelled reference table	14

8.4 Check both models before training	15
8.5 Train the classifier	15
8.6 Classify the observation	15
8.7 Is the observation inside the class it was assigned?	16
8.8 Attributing an OOD result	16
Migration Parameterization	16
Nm (number of migrants) — default	16
m (per-generation rate) — symmetric migration	17
Interactive Model Building	17
References	17

This tutorial demonstrates PipeMaster’s inference pipeline using three demographic models of increasing complexity. All data files are included with the package — no external downloads are needed.

Contents

1. Installation
2. Example Models
3. Computing Observed Statistics
4. Simulating Reference Tables
5. Supervised Machine Learning (SML)
6. Out-of-Distribution Diagnostics
7. Worked Example: PonAbe (Orangutan) RAD-seq, End to End
8. Model Selection: PonAbe IM vs Strict Isolation

1. Installation

Install PipeMaster from GitHub:

```
install.packages("devtools")
devtools::install_github("gehara/PipeMaster")
```

Load the package:

```
library(PipeMaster)
```

PipeMaster requires `ape`, `e1071`, `msm`, and `torch` (installed automatically). `torch` powers all neural-network paths (`tune.nn()`, `nn.predict()`, `classifier`, and OOD diagnostics) and pulls in `luz` as a transitive dependency. For the Shiny GUI, you also need `shiny`, `shinydashboard`, `shinyjs`, and `DT`. For accelerated parallel Hyperband, install `bigmemory` (Suggested); for OOD nearest-neighbor diagnostics, install `RANN` (Suggested).

Simulation engines

PipeMaster includes two coalescent simulation engines:

- **msABC** (built-in C) — a modified version of msABC (Pavlidis et al. 2010), itself based on Hudson’s ms (Hudson 2002). Fast for short loci (RAD-seq, 100 bp). Uses `sim.sumstats()`.
- **scrm** (vendored C++) — a modified version of scrm (Staab et al. 2015), which uses the Sequential Markov Coalescent approximation to efficiently simulate long loci with recombination (WGS, 100 kb). Uses `sim.scrm.sumstats()`.

Both engines compute summary statistics and the site frequency spectrum (SFS) in C during simulation — no intermediate files or parsing overhead.

Parameter scaling

You specify parameters in **natural units** (Ne, generations, per-bp mutation rate). PipeMaster converts them to coalescent units internally:

Natural unit	Coalescent scale	Formula
Population size (Ne)	Relative size	Ne / Ne_0
Time (generations)	Coalescent time	$t / (4 * Ne_0)$
Mutation rate (per bp per gen)	Theta per locus	$4 * Ne_0 * \mu * L$
Migration (Nm)	Scaled migration	$4 * Ne_0 * m$

Where $Ne_0 = 100,000$ is the reference population size used for scaling.

2. Example Models

PipeMaster ships three example models based on well-characterized stdpopsim demographic histories. Each comes with pseudo-observed RAD-seq data (VCF) and pre-computed WGS observed statistics.

2.1 Vaquita two-epoch bottleneck

A single-population bottleneck model for the vaquita porpoise (*Phocoena sinus*) (Robinson et al. 2022). Three parameters.

Parameter	True value	Description
Ne0.pop1	2,807	Current Ne
Ne1.pop1	4,485	Ancestral Ne
t.Ne1.pop1	2,162 gen	Time of size change

Tree: (1); | Constraint: $Ne_0 < Ne_1$ (bottleneck)

2.2 Orangutan isolation with migration

Two-population model for Bornean and Sumatran orangutans (Locke et al. 2011). Eight parameters with asymmetric migration in Nm scale.

Parameter	True value	Description
Ne0.pop1	8,805	Current Bornean Ne
Ne0.pop2	37,661	Current Sumatran Ne
Ne1.pop1	10,617	Ancestral Bornean Ne
Ne1.pop2	7,317	Ancestral Sumatran Ne
Ne.anc_1_2	17,934	Ancestral Ne at split
join1	20,157 gen	Divergence time
mig0.1_2	0.06 Nm	Migration into Bornean
mig0.2_1	0.415 Nm	Migration into Sumatran

Tree: (1,2); | Migration scale: Nm

2.3 Human Out-of-Africa

Three-population model for YRI, CEU, and CHB (Gutenkunst et al. 2009). Fourteen free parameters with symmetric migration in per-generation rate (m).

Parameter	True value	Description
Ne0.pop1	12,300	Current YRI Ne
Ne0.pop2	29,725	Current CEU Ne
Ne0.pop3	54,090	Current CHB Ne
Ne1.pop1	7,300	Ancestral Ne
Ne1.pop2	1,000	Initial CEU Ne at EU/AS split (N_EU0)
Ne1.pop3	510	Initial CHB Ne at EU/AS split (N_AS0)
Ne.anc_3_2	2,100	OoA bottleneck Ne (between EU/AS split and OoA)
join3_2	848 gen	CEU-CHB split time
join2_1	5,600 gen	OoA split time
t.Ne1.pop1	8,800 gen	YRI Ne change time
m_YRI_CEU	2.47e-5	Per-gen rate
m_YRI_CHB	7.67e-6	Per-gen rate
m_CEU_CHB	6.38e-5	Per-gen rate
m_ancestral	1.71e-3	Per-gen rate (YRI-OoA)

Tree: ((3,2),1); | Migration scale: m (symmetric via conditions)

Loading example data

```
library(PipeMaster)

# File paths for Vaquita example
extdir <- system.file("extdata", "Vaquita2Epoch", package = "PipeMaster")
list.files(extdir)
# [1] "model_diffuse.RData"      "model_informative.RData"
# [2] "obs_allstats_wgs.txt"    "pop_assign.txt"
# [3] "Vaquita2Epoch_RAD.vcf"

# Load model (object is named Vaquita2Epoch_WGS_informative)
load(file.path(extdir, "model_informative.RData"))
model <- Vaquita2Epoch_WGS_informative

# File paths
vcf_file <- file.path(extdir, "Vaquita2Epoch_RAD.vcf")
pop_file <- file.path(extdir, "pop_assign.txt")
```

The shipped model is WGS-configured (10,000 loci $\times$ 100,000 bp). If you want to use it with a different locus structure — your own RAD VCF, a PHYLIP alignment, or the bundled `Vaquita2Epoch_RAD.vcf` — align the model to your data first with `get.data.structure()`:

```
# pop.assign must be a data.frame here (unlike observed.sumstats(),
# which also accepts a file path)
pop_assign <- read.table(pop_file, header = TRUE)

# Align model's locus structure to the data (number of loci + locus lengths)
model <- get.data.structure(model,
                           path.to.vcf = vcf_file,
```

```

pop.assign = pop_assign)
# PipeMaster:: 10000 loci declared by the VCF ##contig header
# PipeMaster:: 324 with variants, 9676 invariant (retained)

```

This rewrites `model$loci` and `model$I` to match the data. Skip this step if you're using the pre-computed WGS observed stats below (which were computed against the original WGS model).

Where the locus structure comes from. For VCF input, the locus set and their lengths are read from the `##contig=<ID=...,length=...>` header lines. That header is the authoritative record of what was sequenced — crucially, it lists loci that carry no variants, which the `CHROM` column cannot. Those invariant loci are real data and belong in the per-locus denominator of every summary statistic: a RAD locus that came back monomorphic is evidence of low diversity, not a locus to discard. Dropping them inflates π , θ_W , F_{st} and every other moment.

Supply `chrom.sizes` only if your VCF was written without contig headers, or to override them:

```

model <- get.data.structure(model, path.to.vcf = vcf_file,
                           pop.assign = pop_assign,
                           chrom.sizes = data.frame(chrom = paste0("locus", 1:10000),
                                                    bp = 100))

```

When you supply it, `chrom.sizes` is the callable set — it is used exactly as given, and any contig carrying variants that you did not declare raises a warning naming the count. For PHYLIP input, lengths come from the alignments themselves and neither argument is needed.

3. Computing Observed Statistics

`observed.sumstats()` computes summary statistics and the folded SFS from a VCF or PHYLIP file. It uses the same C stat functions as the simulation engines, ensuring consistency between observed and simulated data.

From a VCF file

```

# Compute observed stats from the shipped RAD VCF
obs <- observed.sumstats(
  model      = model,
  path.to.vcf = vcf_file,
  pop.assign = pop_file
)
cat(sprintf("Observed: %d summary statistics + SFS bins\n", ncol(obs)))

```

Multi-population example (PonAbe)

```

extdir_pa <- system.file("extdata", "PonAbe_TwoSpecies", package = "PipeMaster")
load(file.path(extdir_pa, "model_informative.RData"))
model_pa <- get(ls(pattern = "PonAbe"))

obs_pa <- observed.sumstats(
  model      = model_pa,
  path.to.vcf = file.path(extdir_pa, "PonAbe_TwoSpecies_RAD.vcf"),
  pop.assign = file.path(extdir_pa, "pop_assign.txt")
)

```

Using pre-computed WGS observed stats

WGS VCFs are too large to ship with the package. Pre-computed observed statistics are provided instead:

```
obs_wgs <- read.table(  
  file.path(extdir, "obs_allstats_wgs.txt"),  
  header = TRUE  
)
```

Suggesting prior bounds from observed stats

Before simulating, you can use `suggest.priors()` to derive starting-point prior bounds from the observed summary statistics. The function uses classical population-genetic moment estimators:

- **Ne** from $\pi/(4\mu)$ and $\theta_W/(4\mu)$ (per population)
- **Ancestral Ne** from average of daughter populations
- **Divergence time** from $T = -2*Ne*\log(1 - Fst)$ (pure isolation)
- **Migrants per generation** from $Nm = (1 - Fst)/(4*Fst)$ (drift-migration equilibrium)

The bounds are `[est / expand_factor, est * expand_factor]` (default `expand_factor = 10`, ~2 orders of magnitude around the point estimate).

```
# Compute observed stats first (see above)  
# Then derive priors:  
model <- suggest.priors(  
  model      = model,  
  obs        = obs,          # 1-row data.frame from observed.sumstats()  
  mu         = 5.83e-9,      # per bp per generation  
  expand_factor = 10  
)  
  
# Output (verbose):  
# PipeMaster:: suggest.priors mu=5.83e-09 locus_length=100 expand_factor=10 npops=1 mig_scale=Nm  
# NOTE: these are STARTING-POINT priors from classical moment estimators.  
#       Moment estimators (pi/(4mu), thetaW/(4mu), Fst-based T and Nm)  
#       assume stationarity/equilibrium and are time-averaged. Under  
#       expansion, bottleneck, structure, or selection, they can be  
#       biased by an order of magnitude. Do not trust them blindly:  
#         - run OOD.pretrain() to check obs lies inside the prior support;  
#         - inspect whether the posterior pushes against any prior bound;  
#         - widen expand_factor or set bounds manually if it does.  
# pop 1: Ne_pi=3940 Ne_W=3643 Ne_est=3792 Hd=0.009 TajD=0.004  
# Updated 3 prior rows: Ne0.pop1, Ne1.pop1, t.Ne1.pop1
```

The function writes the suggested `lo/hi` directly into `model$flags` and attaches a `model$prior_suggestions` list with the point estimates, per-pair `Fst` readings, and any warnings.

Caveats (also printed in the NOTE above):

- Moment estimators are **time-averaged**. Under expansion ($Ne_0 \gg Ne_1$), $\pi/(4\mu)$ sits closer to Ne_1 than Ne_0 — the present-day Ne can be 10x larger than the suggested upper bound.
- Tajima's D is reported but **no expansion/bottleneck/constant verdict is made** — $|D| < 0.5$ is too noisy to call, and even larger $|D|$ can reflect selection or structure rather than demography.
- For multi-population models, a single pairwise Fst cannot be decomposed into divergence time and migration. The function reports both readings (T_{iso} under pure isolation, Nm_{eq} under drift-migration equilibrium) and uses `fixed/(fixed+shared)` as a soft tiebreaker, but always issues a per-pair warning. Treat both priors as upper bounds.

Always validate suggested priors with `OOD.pretrain()` (Section 6) before running a large reference table. If the posterior later pushes against any bound, widen `expand_factor` or set the bound manually.

4. Simulating Reference Tables

A reference table contains many simulations under the model with parameters sampled from the prior distribution. Each row contains the sampled parameters and the resulting summary statistics.

RAD-seq reference table (msABC engine)

For short loci (e.g., 100 bp RAD-seq), use `sim.sumstats()`:

```
sim.sumstats(  
  model      = model,  
  nsims      = 10000,      # total number of simulations  
  batch.size = 10,        # simulations per C call (controls memory + progress)  
  mu.rates   = 5.83e-9,   # mutation rate per bp per gen  
  ncores     = 4,         # parallel workers  
  path       = "output/vaquita",  
  output.name = "reftable"  
)  
# Output: output/vaquita/SIMS_reftable.txt
```

`batch.size` controls memory usage and progress-reporting granularity, not the simulation total. For the Vaquita model with 3 parameters, 10,000–50,000 simulations are sufficient.

WGS reference table (scrm engine)

For long loci with recombination (e.g., 100 kb WGS), use `sim.scrm.sumstats()`:

```
sim.scrm.sumstats(  
  model      = model,  
  nsims      = 10000,      # total number of simulations  
  batch.size = 32,        # simulations per C call (scrm default)  
  mu.rates   = 5.83e-9,   # mutation rate  
  rec.rates  = 1e-8,      # recombination rate  
  skip.zns   = TRUE,      # skip ZnS (O(segsites^2)); default TRUE for WGS  
  ncores     = 4,  
  path       = "output/vaquita_wgs",  
  output.name = "reftable"  
)
```

For pathological priors that occasionally generate runaway ARGs, pass `stall.seconds = 600` to enable a head-side worker watchdog that kills any worker whose log file hasn't progressed in that many seconds.

Loading a reference table

```
reftable <- read.table("output/vaquita/SIMS_reftable.txt", header = TRUE)  
  
# Identify parameter and statistic columns  
param_cols <- c("Ne0.pop1", "Ne1.pop1", "t.Ne1.pop1")  
nuisance   <- c("mean.rate", "sd.rate")  
stat_cols  <- setdiff(colnames(reftable), c(param_cols, nuisance))
```

Simulation budget guidelines

Model complexity	Parameters	Recommended sims
1-pop (Vaquita)	3	10,000-50,000
2-pop IM (PonAbe)	8	50,000-100,000
3-pop (OoA)	14-19	100,000-300,000

5. Supervised Machine Learning

PipeMaster’s SML pipeline trains a neural network to learn the mapping from summary statistics to parameters, then predicts parameters for the observed data with uncertainty quantification.

Step 1: Hyperparameter search with `tune.nn()`

```
tune_result <- tune.nn(  
  reftable,  
  param.cols = param_cols,  
  type       = "sumstat",  
  exclude.cols = nuisance,  
  max_epochs = 100,      # max training epochs per model  
  eta        = 3,        # Hyperband reduction factor  
  n_searches = 20,       # parallel Hyperband searches  
  top_k      = 3,        # keep top 3 models as ensemble  
  cores      = 4,        # parallel workers  
  gpus       = 0,        # set to 1+ to use CUDA-capable GPUs  
  seed       = 42,  
  verbose    = TRUE  
)  
  
# Save for later use  
save.tune.result(tune_result, "output/vaquita/tune_result")
```

`tune.nn()` runs Hyperband across `n_searches` parallel workers. With `gpus > 0`, the slowest brackets are routed to GPU workers via `CUDA_VISIBLE_DEVICES`; set `gpu.threshold = N` to switch from CPU to GPU once a Hyperband bracket reaches model budget `N` epochs.

Step 2: Prediction with `nn.predict()`

`nn.predict()` runs one of three uncertainty methods per call. Call it twice — once for the bootstrap interval, once for the ABC-NN-regression posterior — and merge them if you want both.

```
# Locally-weighted residual bootstrap (frequentist prediction interval)  
post_boot <- nn.predict(  
  tune_result,  
  observed = obs_raw,  
  reftable = reftable,  
  param.cols = param_cols,  
  method     = "bootstrap",  
  n_boot     = 5000,      # bootstrap resamples  
  seed       = 42, verbose = TRUE  
)
```



```

# ABC-with-NN-regression (proper Bayesian posterior)
post_abc <- nn.predict(
  tune_result,
  observed = obs_raw,
  reftable = reftable,
  param_cols = param_cols,
  method = "ABC_NN_regression",
  tolerance = 0.05, # ABC acceptance fraction
  seed = 42, verbose = TRUE
)

# Merge into a single posterior object (both slots populated for plotting)
pred <- post_abc
pred$bootstrap <- post_boot$bootstrap
class(pred) <- "posterior"

# Point estimate (weighted ensemble prediction)
pred$point_estimate

# Posterior summary
summary(pred)

```

Two uncertainty methods are available:

- **Residual bootstrap:** Locally-weighted resampling of NN prediction residuals. Captures prediction uncertainty without reference to the prior.
- **ABC-NN regression:** Beaumont et al. (2002) regression adjustment. Proper Bayesian posterior incorporating prior and likelihood.

A third method, `method = "point"`, returns only the ensemble point estimate without uncertainty quantification.

Step 3: Plot posteriors

```
plot(pred, true_values = true_vals, bw.adjust = 2)
```

The `bw.adjust` parameter controls density smoothing (default 1, increase for smoother curves).

Loading saved results

```
tune_result <- load.tune.result("output/vaquita/tune_result")
```

6. Out-of-Distribution Diagnostics

PipeMaster splits OOD checks into two tiers, run at different points in the pipeline:

- **OOD.pretrain()** — prior-predictive coverage. Run before training: is the observed data reachable under the prior + model? Per-stat support, NN density in PCA space, and (when params are passed) NN density in PLS space.
- **OOD.posttrain()** — model-fit + posterior fidelity. Run after `tune.nn()`: NN-latent density (the nonlinear manifold the trained NN uses for regression) and ensemble disagreement at the observed point.

Forensics for either tier: `OOD.projection.diagnose()` (class-dispatched).

Pre-training check (no NN required)

```
pre <- OOD.pretrain(  
  observed = obs_raw,  
  reftable = reftable[, c(param_cols, stat_cols)],  
  stat_cols = stat_cols,  
  param_cols = param_cols,      # enables PLS NN density  
  plot = TRUE  
)  
cat(sprintf("Overall: %s\n", pre$overall))
```

Three checks fire:

1. **Per-stat support** — obs strictly inside / outside each sim range. Outlier fraction drives a tier (pass < 10%, warn 10-25%, fail > 25%).
2. **NN density in PCA space** (all stats + filtered) — distribution-free leave-one-out NN distance, compared to the empirical sim-to-sim NN null.
3. **NN density in PLS space** (all + filtered, when `param_cols` passed) — components aligned with parameter signal.

Post-training check (requires trained NN)

```
post <- OOD.posttrain(  
  trained.nn = tune_result,  
  observed = obs_raw,  
  reftable = reftable[, c(param_cols, stat_cols)],  
  pretrain = pre,      # reuses PCA/PLS context, faster + integrated verdict  
  plot = TRUE  
)
```

Adds:

4. **NN-latent density** — penultimate-layer activations of the top-ranked ensemble model. Catches “curved” out-of-distribution holes that linear PCA/PLS miss.
5. **Ensemble disagreement** — per-param coefficient of variation across the ensemble at obs, compared to the empirical distribution of per-sim mean CVs. High disagreement = model is extrapolating at obs.

Drilling into outliers

```
# Per-outlier sim-distribution histograms  
OOD.outliers(pre, observed = obs_raw, reftable = reftable,  
  pdf_file = "ood_outliers.pdf")  
  
# Projection forensics (PCA/PLS for pretrain; +NN-latent for posttrain).  
# PLS basis also shows param Y-loadings - which params are predicted by  
# each component, paired with which stats load on the same component.  
OOD.projection.diagnose(pre, basis = "pls", pdf_file = "proj_pre.pdf")  
OOD.projection.diagnose(post, basis = "nn_latent", pdf_file = "proj_post.pdf")
```

Diagnosing prior bounds

If priors were set with `suggest.priors()` (Section 3), this is the validation step. Two complementary helpers; use both.

Best-fit boundary pressure — for the simulations that fit obs best (nearest-neighbor posterior in PLS space), where do their params cluster within the prior range? Reuses the cached PLS scores from `OOD.pretrain()`.

```
# Mode A: in-pipeline (uses cached PLS scores, fast)
bf <- OOD.priors.bestfit(ood_result = pre,
                        K_frac = 0.025, edge_threshold = 0.05,
                        basis = "pls",
                        pdf_file = "ood_priors_bestfit.pdf")
print(bf$table[bf$table$edge_pressure != "ok", ])

# Mode B: with an explicit posterior (e.g., from nn.predict ABC-NN regression)
post_abc <- nn.predict(tune_result, observed = obs_raw,
                      reftable = reftable, param_cols = param_cols,
                      method = "ABC_NN_regression", tolerance = 0.05)
bf <- OOD.priors.bestfit(posterior = post_abc, reftable = reftable,
                        pdf_file = "ood_priors_bestfit.pdf")
```

Pairs with `OOD.projection.diagnose(basis = "pls")`: the param Y-loadings panel says “PLS3 wants Ne0+ and join1—”; `OOD.priors.bestfit` confirms whether those params press the upper/lower prior bounds.

Per-outlier marginal consensus — alternative to dropping outlier stats: keep them and widen the priors. For each outlier stat, finds the top-K most-correlated params; translates (`corr sign` × outlier direction) into HI/LO votes; aggregates per param.

```
op <- OOD.priors.outliers(pre, top_k = 3L, corr_threshold = 0.2,
                        pdf_file = "ood_priors_outliers.pdf")
print(op$consensus)           # per-param: n_HI, n_LO, evidence_*, consensus
print(op$per_outlier)        # supporting evidence
```

If `OOD.priors.bestfit()` and `OOD.priors.outliers()` agree on a param/direction, that’s a strong signal to widen. Disagreement is informative too — joint best-fit boundary pressure with no marginal-outlier consensus suggests the prior is too narrow without any single stat calling for it.

Using OOD to filter statistics

```
# Drop outlier statistics before training
ood_pass <- !pre$percentiles$outlier
stat_cols_filtered <- stat_cols[ood_pass]
cat(sprintf("Kept %d/%d statistics\n",
            length(stat_cols_filtered), length(stat_cols)))
```

7. Worked Example: PonAbe (Orangutan) RAD-seq, End to End

This section runs the whole pipeline on the bundled **PonAbe_TwoSpecies** RAD data — Bornean (*Pongo pygmaeus*) and Sumatran (*P. abelii*) orangutans, the isolation-with-migration model from Section 2.2 — with no external downloads. Every number shown below came from an actual run; timings are from an 8-core laptop.

7.1 Load the model and attach the data

```
library(PipeMaster)

extdir <- system.file("extdata", "PonAbe_TwoSpecies", package = "PipeMaster")
```

```
load(file.path(extdir, "model_informative.RData"))
model_IM <- PonAbe_IM_WGS_informative

vcf_file <- file.path(extdir, "PonAbe_TwoSpecies_RAD.vcf")
pop_assign <- read.table(file.path(extdir, "pop_assign.txt"), header = TRUE)

model_IM <- get.data.structure(model_IM, path.to.vcf = vcf_file,
                             pop.assign = pop_assign)
# PipeMaster:: 10000 loci declared by the VCF ##contig header
# PipeMaster:: 5533 with variants, 4467 invariant (retained)
```

The shipped model is WGS-shaped (10,000 loci x 100,000 bp); this rewrites it to the RAD structure (10,000 loci x 100 bp, 20 + 20 haplotypes). Note that 4,467 loci carry no variants and are **kept** — see the discussion in Section 2.

7.2 Observed statistics

```
obs <- observed.sumstats(model_IM, path.to.vcf = vcf_file,
                        pop.assign = pop_assign)
# PipeMaster:: Finished -- 104 sumstats + 441 SFS bins in 2.2 sec

obs[1, c("s_mean_pi", "s_mean_thetaW", "s_mean_Fst")]
#   s_mean_pi  s_mean_thetaW  s_mean_Fst
#   0.17305      0.19429      0.07087
```

7.3 Column bookkeeping

Only 7 of the 11 parameter columns are free. The rest are deterministic consequences of the model's conditions, and feeding them to the network would ask it to learn duplicates:

```
param_cols <- c("Ne0.pop1", "Ne0.pop2", "Ne1.pop1", "Ne1.pop2",
               "join1", "mig0.1_2", "mig0.2_1")

# Derived, not free:
#   t.Ne1.pop1 = t.Ne1.pop2 = t.Ne.anc_1_2 = join1    (the "=" time conditions)
#   Ne.anc_1_2 = Ne1.pop1 + Ne1.pop2                  (model$sum_anc_ne == TRUE)
derived <- c("Ne.anc_1_2", "t.Ne1.pop1", "t.Ne1.pop2", "t.Ne.anc_1_2")
nuisance <- c("mean.rate", "sd.rate")
```

Verify rather than trust — the conditions are model-specific:

```
all(reftable$t.Ne1.pop1 == reftable$join1)           # TRUE
all(reftable$Ne.anc_1_2 == reftable$Ne1.pop1 + reftable$Ne1.pop2) # TRUE
```

7.4 Reference table

Two budgets. Pick by what you are doing:

Tier	sims / model	4 cores	8 cores	Use for
Live (in class)	2,000	~4 min	~2 min	Seeing the pipeline run
Production	50,000	~1.7 h	~40 min	Results you would report

Measured throughput on this model: ~30,600 sims/h on 4 cores, ~73,300 sims/h on 8 cores (10,000 loci x 100 bp, 40 haplotypes).

```

sim.sumstats(model_IM, nsims = 2000, batch.size = 8,
             mu.rates = 2e-8, ncores = 8,
             path = "output/ponabe", output.name = "IM")

reftable <- read.table("output/ponabe/SIMS_IM.txt", header = TRUE)
dim(reftable)  # 2000 x 558 = 11 params + 2 nuisance + 545 stats

stat_cols <- setdiff(colnames(reftable), c(param_cols, derived, nuisance))
length(stat_cols)  # 545 = 104 summary statistics + 441 folded SFS bins

obs_raw <- setNames(as.numeric(obs[1, stat_cols]), stat_cols)

```

The live tier is **under-powered on purpose** and the diagnostics will say so: at 2,000 simulations for 7 parameters, expect `OOD.posttrain()` to return `warn` with high ensemble disagreement. That is the tool working correctly, not a failure to hide. Treat live-tier posteriors as a demonstration of mechanics, never as an estimate.

7.5 Prior check before you spend compute

```

pre <- OOD.pretrain(observed = obs_raw,
                   reftable = reftable[, c(param_cols, stat_cols)],
                   stat_cols = stat_cols, param_cols = param_cols,
                   plot = TRUE)
pre$overall      # "pass"
sum(pre$percentiles$outlier)  # 31 of 545 stats outside sim support

```

Note the accessor: `OOD.pretrain()` and `OOD.posttrain()` report their verdict in `$overall`, while the classifier variants in Section 8 use `$verdict`.

7.6 Train and predict

```

tune_result <- tune.nn(reftable, param_cols = param_cols, type = "sumstat",
                     exclude.cols = c(derived, nuisance),
                     max_epochs = 100L, eta = 3L,
                     n_searches = 4L, top_k = 3L,
                     cores = 4L, gpus = 0L, seed = 42, verbose = TRUE)
save.tune.result(tune_result, "output/ponabe/tune_IM")

post_boot <- nn.predict(tune_result, observed = obs_raw, reftable = reftable,
                      param_cols = param_cols, method = "bootstrap",
                      n_boot = 5000L, seed = 42)
post_abc <- nn.predict(tune_result, observed = obs_raw, reftable = reftable,
                     param_cols = param_cols, method = "ABC_NN_regression",
                     tolerance = 0.05, seed = 42)

pred <- post_abc
pred$bootstrap <- post_boot$bootstrap
class(pred) <- "posterior"

summary(pred)
plot(pred, bw.adjust = 2)

```

`tolerance = 0.05` keeps 5% of simulations. At the live tier that is only 100 rows — raise it to 0.2 for a usable ABC posterior at small budgets, and drop back to 0.05 at production scale.

7.7 Post-training diagnostics

```
post <- OOD.posttrain(trained.nn = tune_result, observed = obs_raw,
                     reftable = reftable[, c(param_cols, stat_cols)],
                     pretrain = pre, plot = TRUE)
post$overall
```

8. Model Selection: PonAbe IM vs Strict Isolation

The question behind the PonAbe model is whether the data require gene flow at all. PipeMaster answers it by simulating both models, training a classifier to tell them apart, and asking which one the observation looks like.

8.1 Build the competing model

Strict isolation is the IM model with migration removed. Nothing else changes — same priors, same divergence time, same locus structure — so the comparison isolates one factor:

```
model_ISO <- model_IM
model_ISO$flags$m <- NULL
model_ISO$labels$name <- "ISO"
```

Confirm the ms command really lost the flag:

```
grepl(" -m ", PipeMaster::msABC.commander(model_IM, arg = 1)[[1]]) # TRUE
grepl(" -m ", PipeMaster::msABC.commander(model_ISO, arg = 1)[[1]]) # FALSE
```

8.2 Simulate the second reference table

```
sim.sumstats(model_ISO, nsims = 2000, batch.size = 8,
              mu.rates = 2e-8, ncores = 8,
              path = "output/ponabe", output.name = "ISO")

rt_IM <- read.table("output/ponabe/SIMS_IM.txt", header = TRUE)
rt_ISO <- read.table("output/ponabe/SIMS_ISO.txt", header = TRUE)

ncol(rt_IM)      # 558
ncol(rt_ISO)     # 556 -- two fewer: no mig0.1_2, mig0.2_1
```

Use **equal numbers of simulations per model**. The classifier learns the class balance you give it, and an unbalanced reference table biases the model probabilities directly.

8.3 Stack into a labelled reference table

The two reftables have different parameter columns, so stack on the shared statistics plus a label column:

```
rt_both <- rbind(
  data.frame(model = "IM", rt_IM[, stat_cols], check.names = FALSE),
  data.frame(model = "ISO", rt_ISO[, stat_cols], check.names = FALSE))

table(rt_both$model)
#  IM  ISO
# 2000 2000
```

8.4 Check both models before training

```
pre_c <- OOD.pretrain.classify(reftable = rt_both, model_col = "model",
                             observed = obs_raw, stat.cols = stat_cols,
                             plot = TRUE)
pre_c$verdict                # "pass"
sum(pre_c$universal_mask)    # 29 of 545 stats rejected by BOTH models
```

The key output is `universal_mask`: statistics whose observed value falls outside *every* candidate model's support. Those cannot discriminate — no model explains them — so they are noise for the classifier. Statistics rejected by only *some* models are exactly the informative ones. If the universal fraction is large the verdict fails, and the honest response is a better model, not a better classifier.

8.5 Train the classifier

```
clf <- tune.nn.classify(rt_both, model_col = "model",
                      max_epochs = 100L, eta = 3L,
                      n_searches = 4L, cores = 4L, gpus = 0L,
                      seed = 42, verbose = TRUE)
save.classifier(clf, "output/ponabe/clf")

clf$metrics$accuracy      # 0.99
clf$metrics$confusion
#           pred:IM pred:ISO
# true:IM      51      0
# true:ISO      0      51
```

High accuracy here means the two models are **distinguishable given this data structure** — a prerequisite for the comparison being meaningful, and worth reporting. If accuracy sits near 0.5 the models are not identifiable from these statistics and no probability you compute afterwards means anything.

8.6 Classify the observation

Run all three output rules. They answer subtly different questions:

```
for (m in c("softmax", "mahalanobis", "latent_nn")) {
  ms <- nn.predict.classify(clf, observed = obs_raw, method = m,
                           reftable = rt_both, model_col = "model")
  cat(sprintf("%-12s -> %s\n", m, ms$best_model))
}
# softmax      -> IM      P(IM) = 0.992
# mahalanobis  -> IM      P(IM) = 1.000
# latent_nn    -> IM      P(IM) = 1.000
```

- **softmax** — the classifier's native output. Measures which class *direction* the observation aligns with in latent space. It will always name a winner, even for an observation lying far outside both classes.
- **mahalanobis** — distance to each class centroid using the pooled within-class covariance. Density-aware, and the one to trust when the observation may be out of distribution.
- **latent_nn** — distance to the nearest simulation of each class. Purely geometric.

Agreement across all three is the result you want, and it is what the orangutan data give: the pseudo-observed data were generated under the IM model, and all three rules recover it. Disagreement is a warning that the observation sits somewhere the classifier is extrapolating — check `OOD.posttrain.classify()` before believing any of them.

8.7 Is the observation inside the class it was assigned?

A model probability only means something if the observation actually lies within the winning class’s learned manifold:

```
post_c <- OOD.posttrain.classify(clf, observed = obs_raw, reftable = rt_both,
                                model_col = "model", pretrain = pre_c,
                                plot = TRUE)

post_c$verdict      # "pass"
post_c$predicted_class # "IM"
post_c$margin       # 0.984 = P(top1) - P(top2)
```

Three outcomes, and only one of them lets you report the probability:

Verdict	Meaning
pass	Observation lies inside the predicted class. Probability is usable.
warn	Inside <i>some</i> class, but not the predicted one. Prediction unreliable.
fail	Outside every class. The classifier is extrapolating; the probability is meaningless.

A **fail** here is the common and important case with real data: it means none of your candidate models can produce the observed data, so the “winner” is only the least-bad of a bad set. Add a model rather than report the ranking.

8.8 Attributing an OOD result

When `OOD.posttrain.classify()` returns **fail**, this attributes the out-of-distribution-ness to individual statistics via integrated gradients:

```
OOD.attribute.classify(clf, observed = obs_raw, reftable = rt_both,
                       model_col = "model",
                       targets = c("centroid", "nearest", "logit"),
                       plot = TRUE)
```

Reading the heatmaps: which SFS bins or summary statistics push the observation away from each class centroid tells you what the missing model ingredient looks like — an excess of rare variants suggests growth, a deficit of shared polymorphism suggests less gene flow than any candidate allows.

Migration Parameterization

PipeMaster v2 supports two migration scales, selected via `model$mig_scale`:

Nm (number of migrants) — default

Migration priors specify Nm, the number of migrants per generation into the receiving population. This is population-size-dependent: the same Nm implies different per-generation rates for populations with different Ne.

```
model$mig_scale # "Nm"
model$flags$m   # migration priors in Nm units
```


m (per-generation rate) — symmetric migration

For models with symmetric migration (e.g., Gutenkunst 2009), use `model$mig_scale = "m"`. Both directions share the same prior and are constrained equal:

```
model$mig_scale <- "m"
# In model$conds$mig:
#   mig0.1_2 = mig0.2_1 (symmetric YRI-CEU)
#   mig0.1_3 = mig0.3_1 (symmetric YRI-CHB)
```

Interactive Model Building

PipeMaster includes a Shiny GUI for interactive model building:

```
main.menu.gui()
```

The GUI provides: - Population tree builder with drag-and-drop - Prior distribution specification with real-time validation - Migration rate configuration (Nm or m scale) - Condition constraints (parameter ordering, equality) - Live demographic plot with colorblind-safe palettes - Export to R object, file, or PDF figure

References

- Gutenkunst R.N. et al. (2009) Inferring the joint demographic history of multiple populations from multidimensional SNP frequency data. *PLoS Genetics* 5: e1000695.
- Locke D.P. et al. (2011) Comparative and demographic analysis of orang-utan genomes. *Nature* 469: 529-533.
- Robinson J.A. et al. (2022) The critically endangered vaquita is not doomed to extinction by inbreeding depression. *Science* 376: 635-639.
- Beaumont M.A. et al. (2002) Approximate Bayesian computation in population genetics. *Genetics* 162: 2025-2035.
- Hudson R.R. (2002) Generating samples under a Wright-Fisher neutral model of genetic variation. *Bioinformatics* 18: 337-338.
- Pavlidis P., Laurent S. & Stephan W. (2010) msABC: a modification of Hudson's ms to facilitate multi-locus ABC analysis. *Molecular Ecology Resources* 10: 723-727.
- Staab P.R. et al. (2015) scrm: efficiently simulating long sequences using the approximated coalescent with recombination. *Bioinformatics* 31: 1680-1682.